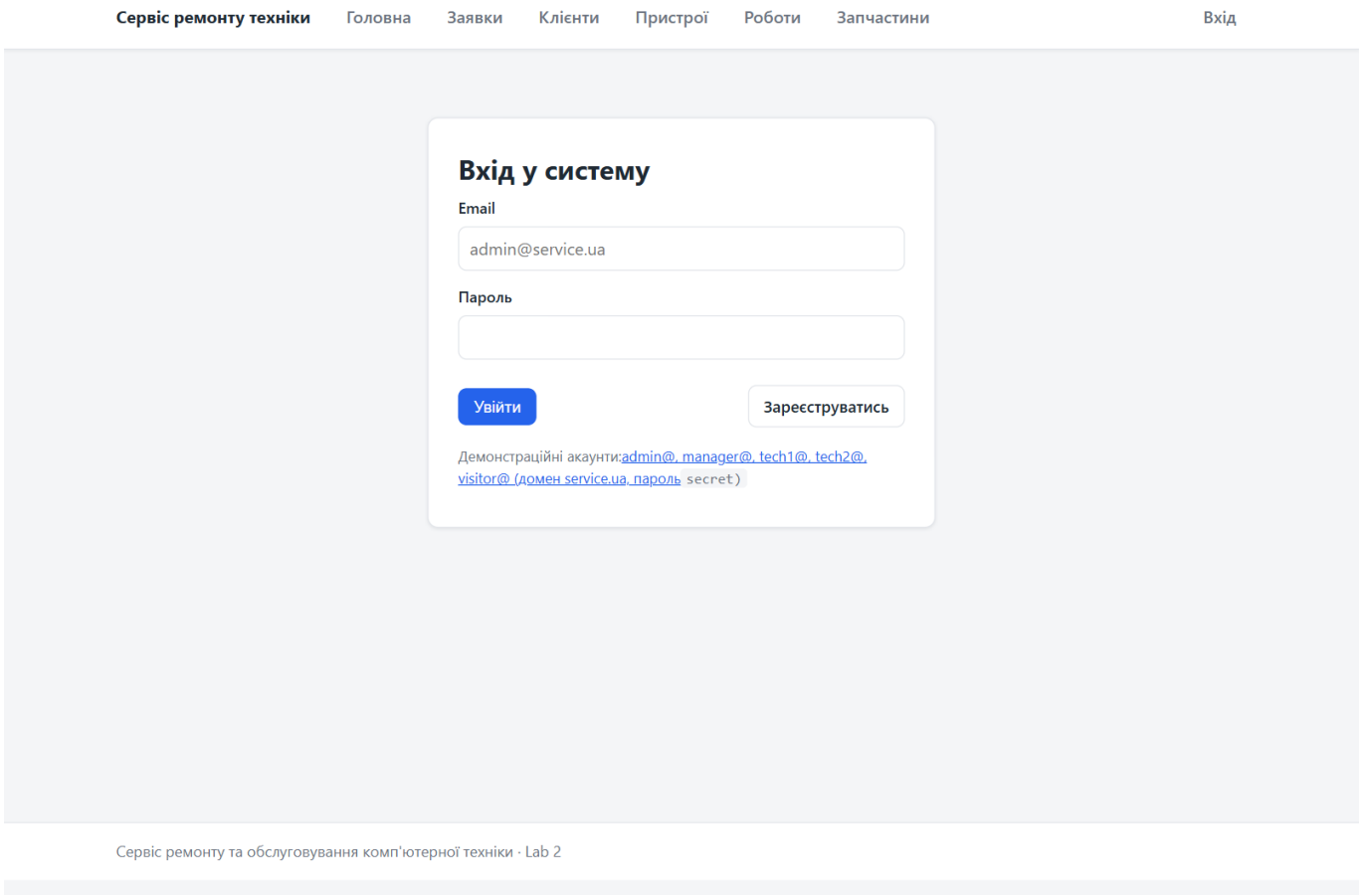
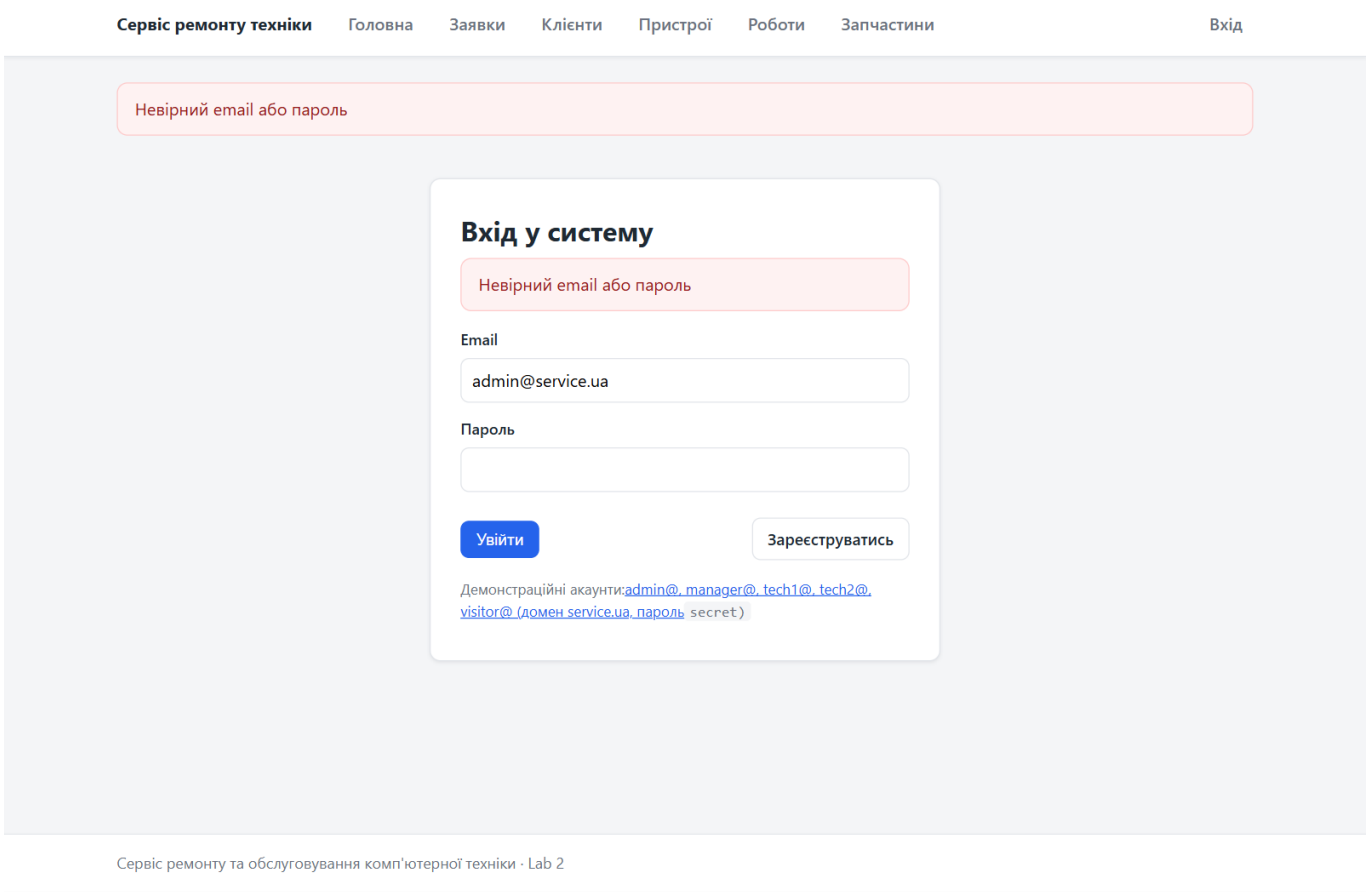
ЗВІТ
про виконання лабораторної роботи №4
«Авторизація та контроль доступу»
з дисципліни
«Вебтехнології»
Студента групи КН-2327Б
Багрія-Белза Андрія

Мета роботи: Реалізувати авторизацію користувачів (Passport.js, express-session, bcryptjs) та контроль доступу за ролями: четверта роль Відвідувач, захист мутуючих маршрутів, сторінка 403.

Завдання 1: Вхід та вихід користувачів: форма /login (email + пароль), локальна стратегія Passport.js, перевірка хеша bcryptjs, сесії express-session; публічна реєстрація з присвоєнням ролі Відвідувач.





Форма входу та відмова за невірними даними: статус 401 і повторний рендер форми з банером «Невірний email або пароль». У сесії зберігається лише id користувача (serializeUser/deserializeUser), а роль читається з БД при кожному запиті, тому зміна ролі в адмінці діє негайно. Cookie сесії: httpOnly, sameSite=lax, живучість 8 годин.

Завдання 2: Захист маршрутів за ролями: middleware requireRole(roles) — неавторизованого редіректить на /login?next=..., авторизованого без потрібної ролі — на сторінку 403.

1. src/lib/auth.ts — middleware захисту маршруту

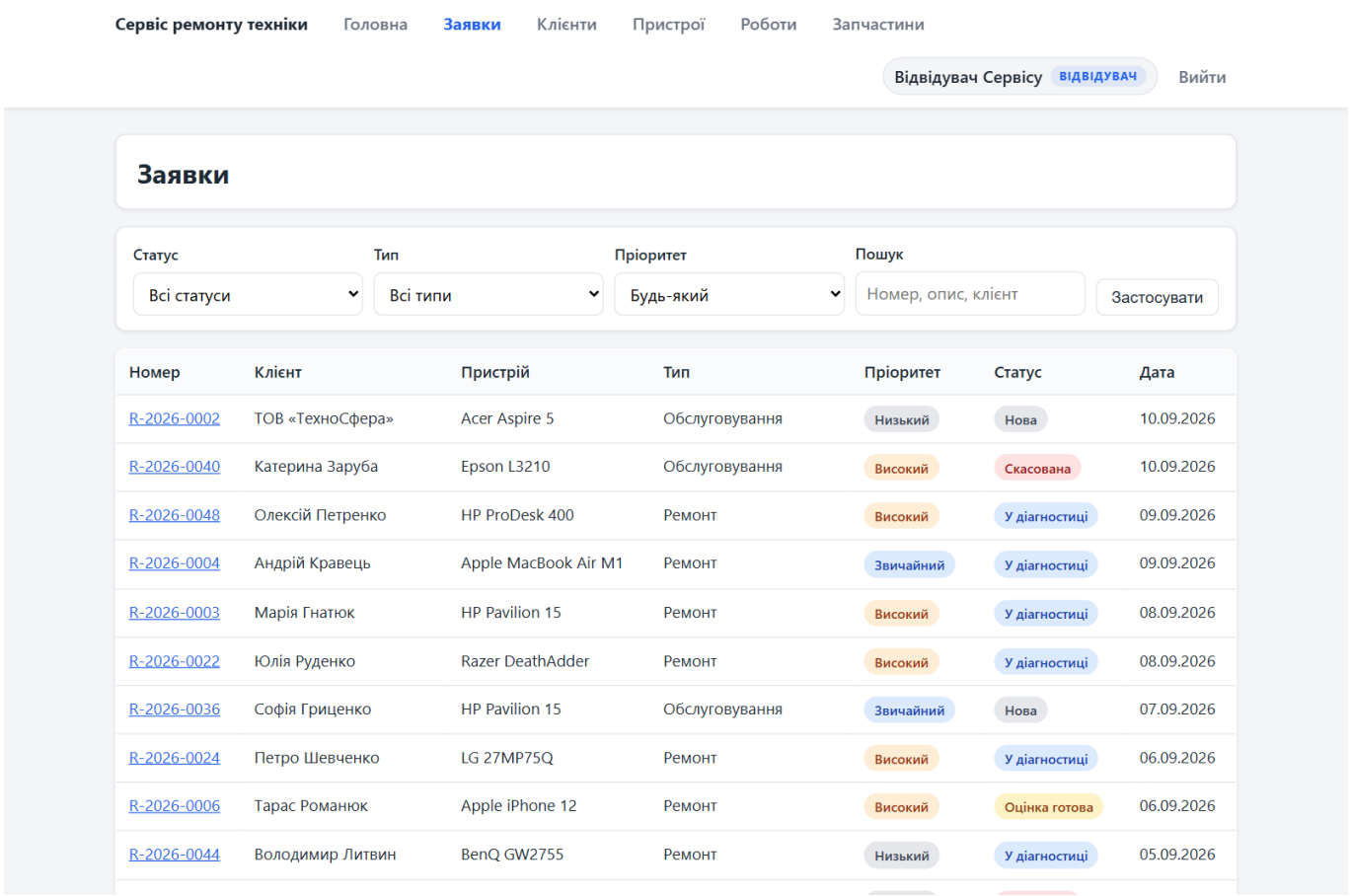
```
export function requireRole(roles: readonly string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) {
      const nextUrl = req.originalUrl;
      return res.redirect(`/login?next=${encodeURIComponent(nextUrl)}`);
    }
    if (!roles.includes(req.user.role)) {
      return res
        .status(403)
        .render("errors/403", {
          nav: "none",
          requiredRoles: roles.map((r) => ROLE_LABELS[r] ?? r).join(", "),
        });
    }
    next();
  };
}
```

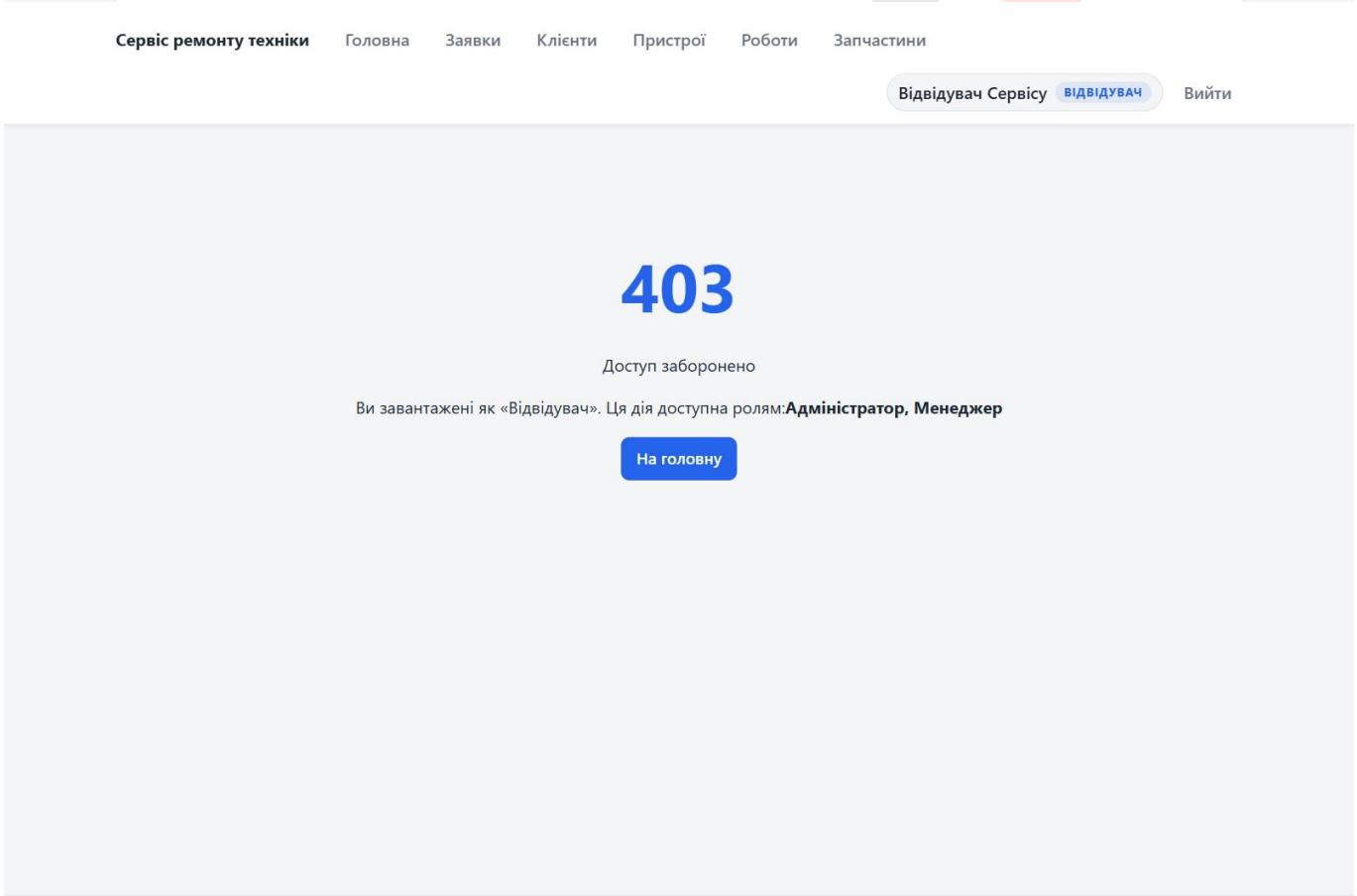
2. src/routes/clients.ts — матриця дозволів на маршрутах

```
import { requireRole, ROLES } from "../lib/auth.js";

// Матриця дозволів (Lab 4): клієнти веде менеджер і адміністратор.
const router = Router();
router.get("/", index);                                  // перегляд – всі
router.get("/new", requireRole(ROLES.MANAGER), createForm);
router.post("/", requireRole(ROLES.MANAGER), create);
router.get("/:id", show);
```

```
router.get("/:id/edit", requireRole(ROLES.MANAGER), editForm);
router.post("/:id/edit", requireRole(ROLES.MANAGER), edit);
router.post("/:id/delete", requireRole(ROLES.MANAGER), remove);
```





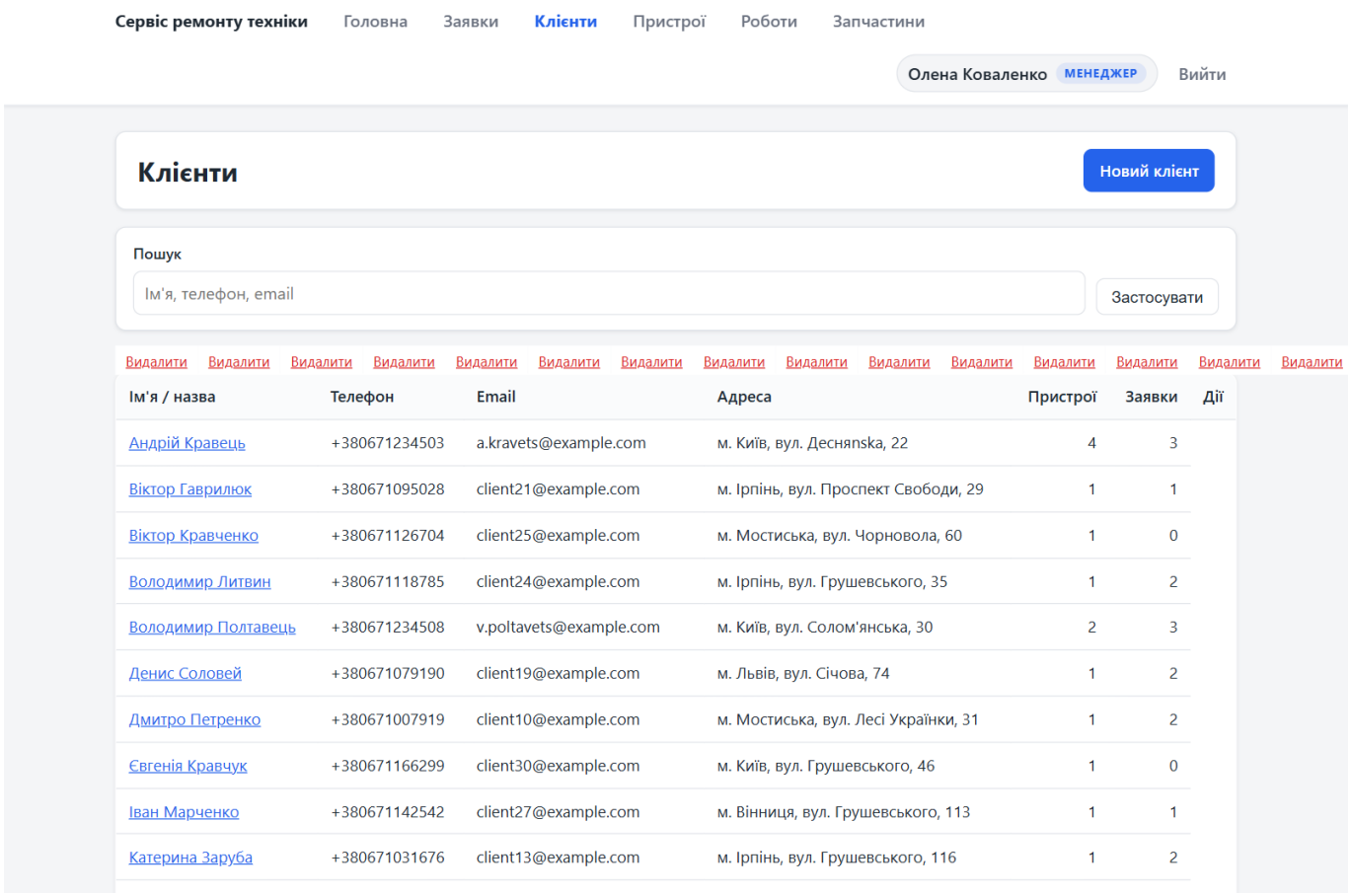
Відвідувач (роль VISITOR) має лише перегляд: список заявок відкривається без входу та після входу, а спроба перейти на створення заявки завершується сторінкою

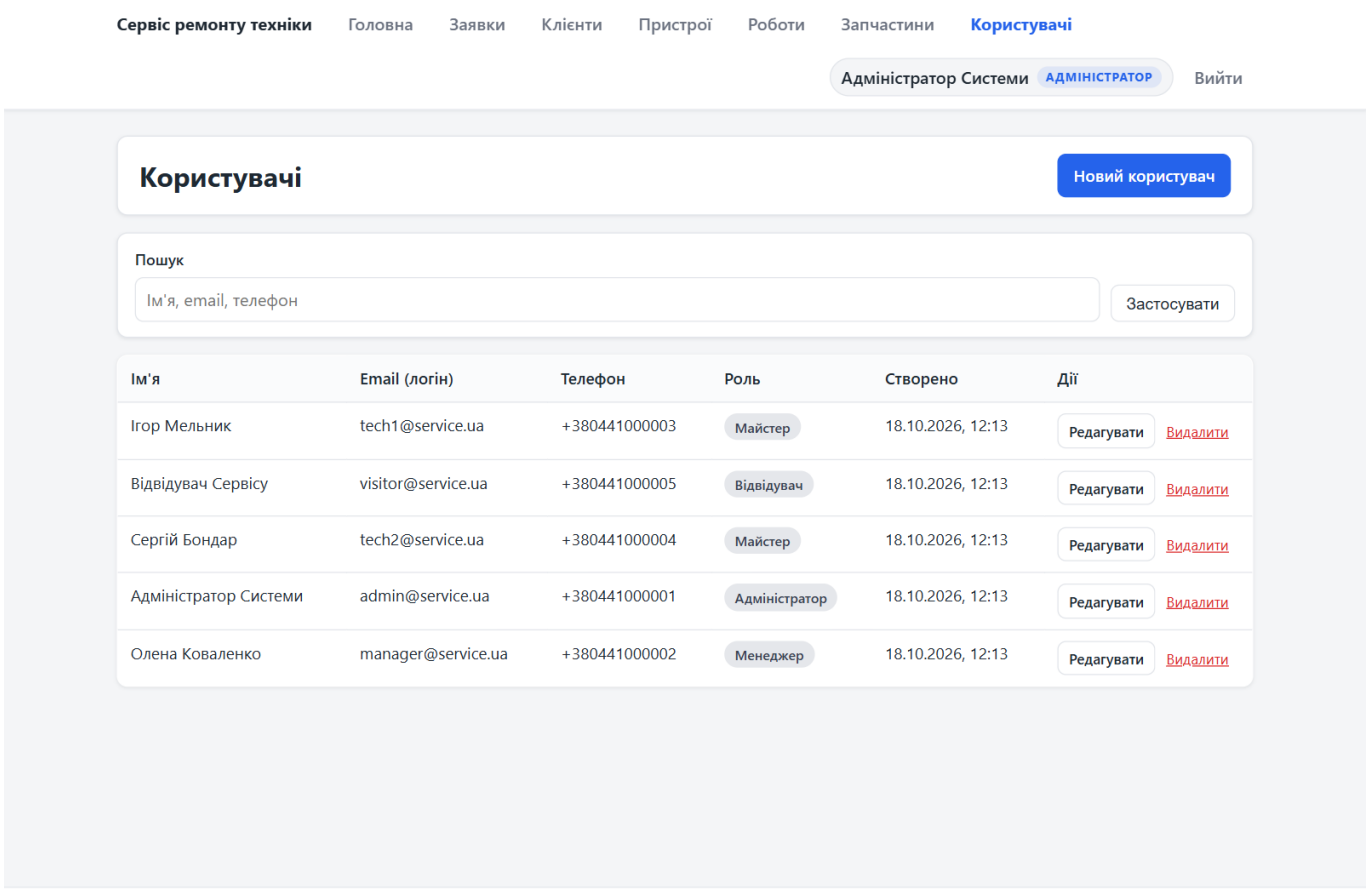
403, де названо, яка роль потрібна. Гість натомість отримує редірект на /login?next=... і після входу повертається на цю саму сторінку.

Завдання 3: Матриця дозволів для 4 ролей і role-aware інтерфейс: меню й дії відображаються лише тим, хто може їх виконати; фінальний захист — завжди на сервері.

1. src/lib/auth.ts — групи ролей

```
export const ROLES = {
  /** Майстер і вище (операції зі статусом/смігом заявок). */
  TECH: ["ADMIN", "MANAGER", "TECHNICIAN"],
  /** Менеджер і вище (довідники, фінанси). */
  MANAGER: ["ADMIN", "MANAGER"],
} as const;
```





Права наростають за ролями: Відвідувач — лише перегляд; Майстер — статус заявок, роботи/запчастини, діагностика, коментарі; Менеджер — повне управління заявками, клієнтами, пристроями, каталогами та фінансами; Адміністратор — усе, включно з модулем користувачів (Lab 5). На скріншотах — список клієнтів від імені менеджера та модуль користувачів, доступний лише адміністратору.

Висновок: На цій лабораторній роботі я реалізував авторизацію користувачів за допомогою Passport.js (локальна стратегія email+пароль, bcryptjs) та сесій express-session, міграцією додав четверту роль Відвідувач і заклав її в seed. Захист маршрутів зроблено middleware requireRole(roles): гостя редіректить на /login?next=..., а авторизованого користувача без потрібної ролі — на сторінку 403 із зазначенням, яка роль потрібна. Усі чотири ролі перевірено на живому застосунку: відвідувач бачить лише перегляд, майстер керує роботою над заявками, менеджер — довідниками та фінансами, адміністратор — усе включно з користувачами. Закріпив механіку сесій (httpOnly-cookie, serialize/deserialize, роль з БД на кожному запиті) і паттерн «одна матриця дозволів на всіх мутуючих маршрутах».